

PARTNER INTEGRATION GUIDE



Integrating the Amen Platform API

A practical handbook for partner developers: from API key to your first completed escrow deal, using the official Amen API Integration Starter Kit as your reference.

Audience

Developers building on the Amen API

Reference code

github.com/KMG411/amen-integration-starter-kit

Version

1.0 · API v1

CONTENTS

1. Start here	7. Errors & retries
2. How an Amen integration works	8. Mobile apps & web dashboards
3. Your first deal in 10 minutes	9. Go-live checklist
4. Copying the reference into your codebase	10. Troubleshooting
5. Payments	Appendix A. Endpoint map & deal statuses
6. Webhooks	Appendix B. Postman, versioning & support

01 Start here

The **Amen API Integration Starter Kit** is Amen's official reference repository. It contains a small, working, copy-ready integration of the Amen Platform API in every major technology stack — all following one pattern and one reference scenario. Use it as the model for your own integration rather than starting from the raw API reference.

Repository	https://github.com/KMG411/amen-integration-starter-kit (MIT licence — copy freely into proprietary code)
API reference	https://docs.amnn.sa · spec vendored at openapi/openapi.yml
Sandbox	https://sandbox-api.amnn.sa — safe to create test customers, deals and payments
Live	https://api.amnn.sa — real money; separate token
Authentication	header <code>X-API-Token: <token></code> on every request; tokens are environment-specific

Pick your stack

Stack	Folder	Run the reference scenario
Python 3.11	<code>python/</code>	<code>pip install -e ".[dev]" && python examples/01_golden_path.py</code>
TypeScript / Node 20	<code>typescript/</code>	<code>npm install && npm run example:golden-path</code>
JavaScript / Node 20	<code>javascript/</code>	<code>npm install && npm run example:golden-path</code>
PHP 8.1+ (Laravel-ready)	<code>php/</code>	<code>composer install && php examples/01_golden_path.php</code>
Java 17 (Maven)	<code>java/</code>	<code>mvn -q compile exec:java</code>
C# / .NET 8	<code>csharp/</code>	<code>dotnet run --project examples/GoldenPath</code>
Kotlin (Gradle) · Android notes	<code>kotlin/</code>	<code>gradle :golden-path:run</code>

Stack	Folder	Run the reference scenario
Flutter / Dart 3	flutter/	cd amen_client && dart pub get && dart run example/golden_path.dart
Swift 5.9 (iOS/macOS)	swift/	swift run golden-path
Postman	postman/	Import the collection + <i>Amen</i> — <i>Sandbox</i> environment, set <code>apiToken</code> , run the folder in order

Every stack folder has the same layout

```

<stack>/
├── README.md                ← 3 commands to run it
├── src/amen/
│   ├── client              ← AmenClient: auth header, base URL, timeouts, retries
│   ├── config              ← reads the four AMN_* environment variables
│   ├── models              ← Deal, Customer, Withdrawal, Webhook ... from the spec
│   ├── errors              ← AmenApiError { status, codes[], has(code) }
│   ├── resources/         ← lookups · account · customers · deals(.actions) · withdrawals · webhooks
│   └── webhooks/          ← signature verification + framework-agnostic handler
├── examples/              ← 01_golden_path · 02_cancel_and_dispute · 03_withdrawal · 04_webhook
└── tests/                 ← unit (offline) · integration (sandbox, needs AMN_API_KEY)

```

Configuration convention. All stacks read the same four environment variables and fall back to a `.env` file in the stack folder or the kit root. Copy `.env.example` to `.env` , set `AMN_API_KEY` , and never commit `.env` .

02 How an Amen integration works

Amen is an **escrow service**: the buyer pays into Amen, Amen holds the funds while the seller delivers, and releases them when the deal completes. Your platform creates the parties and the deal, drives the lifecycle through *actions*, and reacts to status changes.

The three objects you work with

Customer

A buyer or seller: name + phone. Identified by `customer_number` (e.g. `CUST-0001`). Create once, reuse across deals.

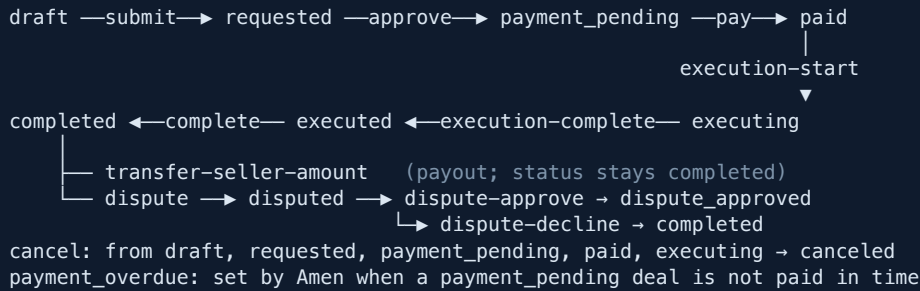
Deal

The escrow transaction: an **offer** (product or service), price, parties, addresses. Identified by `deal_number` . Has a `status` that only actions can change.

Account & wallet

Your merchant account: wallet balance (available / escrow / on-hold), linked bank accounts, withdrawals.

Deal lifecycle



Only `draft` deals can be edited or deleted. Submitting requires a buyer **and** a seller; product deals also need a delivery address. Every reference client checks these rules **locally** before calling the API and raises `AmenLifecycleError` with the allowed statuses — so you find ordering mistakes in development, not in production.

The reference scenario ("golden path")



Defined once in `scenario/golden-path.yml`; every stack's `01_golden_path` example and integration test implement it step for step. Secondary scenarios (cancel, dispute, withdrawal) are in `02_...` and `03_...`.

Deeper: `docs/02-deal-lifecycle.md` · `scenario/golden-path.yml`

03 Your first deal in 10 minutes

1. **Get a sandbox token** from the Amen platform and clone the kit: `git clone https://github.com/KMG411/amen-integration-starter-kit.git`
2. **Configure:** `cp .env.example .env` and set `AMN_ENV=sandbox`, `AMN_API_KEY=...`
3. **Check the connection:** `curl -H "X-API-Token: $AMN_API_KEY" https://sandbox-api.amnn.sa/api/v1/account` — you should see your account and wallet.
4. **Run the golden path** in your stack (table in §1). Expected output:

```
environment: sandbox (https://sandbox-api.amnn.sa)
```

- ✓ customers CUST-0101 (buyer), CUST-0102 (seller)
- ✓ deal DL-000245 created → status=draft
- ✓ parties → status=draft
- ✓ delivery address → status=draft
- ✓ submit → status=requested
- ✓ approve → status=payment_pending
allowed payment methods: [wallet, mada, visa_master, applepay]
- ✓ pay with wallet → status=paid
- ✓ execution-start → status=executing
- ✓ execution-complete → status=executed
- ✓ complete → status=completed
- ✓ transfer-seller-amount (payout) → status=completed

🎉 deal DL-000245 finished: completed

If the run stops at payment_pending with NEEDS_TOP_UP : the sandbox buyer wallet has no balance. Either top it up (bank details are in `GET /api/v1/account → wallet.top_up_account`) or complete the online checkout it created, then re-run the example with the deal number as an argument to resume from `paid`.

5. **Read the example** you just ran — it is under 80 lines and is the pattern for your own code.
6. **Run the tests:** unit tests are offline; integration tests repeat the golden path against the sandbox and skip automatically without a token.

04 Copying the reference into your codebase

The kit is deliberately small so you can lift it, not depend on it. Recommended approach: copy the `src/amen` folder of your stack into your project and adapt the three places that are yours to decide.

Copy as-is

- **AmenClient** — one place for the token header, base URL, timeouts, and retry policy (only 429 and 5xx are retried, with jittered backoff and `Retry-After`).
- **Resources** — `customers`, `deals`, `deals.actions`, `withdrawals`, `webhooks`, `lookups`, `account`, each mirroring one API tag.
- **Lifecycle guard** — the `ALLOWED_FROM` table and the check before every action.
- **AmenApiError** — parsed error codes with `has(code)`.
- **Webhook verification + handler**.

Adapt to your project

- **Configuration** — replace `.env` loading with your secret manager (Vault, AWS/Azure/GCP secrets, Laravel config, Spring properties...). Keep the four variable names.
- **Logging** — plug in your logger; never log `X-API-Token` or webhook secrets.
- **Persistence** — store `deal_number`, `customer_number` and the last known `status` in your database; treat Amen as the source of truth and re-read on doubt.
- **Idempotency store** — the handlers keep seen webhook ids in memory; back it with Redis or your DB in production.

Conventions to keep

Topic	Rule
Money	Always a string with two decimals in SAR: <code>"1000.00"</code> . Never floats. The examples show the helper pattern.
Timestamps	API values are epoch milliseconds ; the models convert them to native date types.
Identifiers	Use <code>number</code> fields (<code>DL-...</code> , <code>CUST-...</code> , <code>WR-...</code>) in URLs and your own records; <code>id</code> is opaque.
Pagination	List endpoints are paged. Use the <code>iterAll</code> / <code>all()</code> helpers rather than reading page 0 only.
Unknown fields	All models ignore unknown JSON fields, so additive API changes never break you. Keep that behaviour.
Mutating calls	The client sends <code>Origin</code> / <code>Referer</code> on POST/PUT/DELETE; keep it — the API applies origin checks.

Never put the API token in a mobile or browser app. It can be extracted from any binary or bundle. Mobile and dashboard front ends call *your* back end, which holds the token (see §8).

Deeper: `CONTRIBUTING.md` (naming conventions) · `docs/06-security-checklist.md` · `docs/09-common-mistakes.md`

05 Payments

- After `approve` the deal is `payment_pending`. Call `GET /deals/{n}/allowed-payment-methods/` to learn which methods apply: `wallet`, `mada`, `visa_master`, `applepay`.
- Wallet** — `POST action/make-payment-wallet`. Synchronous: the deal becomes `paid` immediately if the buyer wallet has funds.
- Online (card / Apple Pay)** — `POST action/make-payment-online { "payment_method": "mada" }` returns a **Checkout** (`provider: hyperpay`, `hyperpay.checkout_id`). Render HyperPay's checkout widget/SDK with that id in your UI. Amen marks the deal `paid` when the provider confirms — wait for the webhook or poll `GET /deals/{n}`.
- Service deals: pass the final `price` at `approve` time (`price__required` otherwise).
- Payout to the seller is explicit: `action/transfer-seller-amount` after `completed`. Withdraw your own available balance with `POST /withdrawals/` to a linked bank account.

Deeper: `docs/03-payments.md` · examples `01_golden_path`, `03_withdrawal`

06 Webhooks

Register a URL once; Amen calls it on deal events. The reference receivers implement the five rules below — copy them exactly.

Register	<code>POST /api/v1/web-hooks/ { "url": "https://your-host/webhook" } → { id, url, secret_key }</code>
Secret	<code>secret_key</code> is returned only at creation . Store it immediately as <code>AMN_WEBHOOK_SECRET</code> in your secret manager. If lost, delete and re-create the webhook.
Receiver	Every stack ships <code>examples/04_webhook_receiver</code> (Express, Flask-free stdlib, PHP built-in server, JDK HttpServer, ASP.NET minimal API, ...) and a framework-agnostic <code>WebhookHandler</code> .

The five rules

1. **Read the raw body first.** Verify the signature over the exact bytes received — never over a re-serialised object. (Express: `express.raw()`; Laravel: `$request->getContent()`; ASP.NET: copy `Request.Body`.)
2. **Verify, then parse.** Reject with 401 on mismatch. The verifier accepts hex digests, optionally prefixed `sha256=`, and compares in constant time.
3. **Respond 200 quickly.** Queue any real work; deliveries that time out are retried.
4. **De-duplicate by event id.** Deliveries can repeat; the handler keeps a seen-set (back it with Redis/DB in production).
5. **Don't trust the payload for money decisions.** On any event that matters, re-read the deal with `GET /deals/{n}` before updating your records.

```
// TypeScript – the same shape exists in every stack
const handler = new WebhookHandler(config.webhookSecret, async (event) => {
  const deal = await amen.deals.get(event.data.deal_number); // re-read, then act
  await orders.updateStatus(deal.number, deal.status);
});
app.post("/webhook", express.raw({ type: "*" }), async (req, res) => {
  const { status, body } = await handler.handle(req.headers, req.body); // raw bytes
  res.status(status).json(body);
});
```

Signature header and payload schema. The verifier's header name and algorithm are a single configuration point (`SIGNATURE_HEADER` / `ALGORITHM` in each stack's `webhooks/verify`), defaulting to HMAC-SHA256 of the raw body. Confirm the current values and event catalogue with the Amen integration team; `docs/04-webhooks.md` is updated as they are published.

Deeper: `docs/04-webhooks.md` · `<stack>/src/amen/webhooks/` · `examples/04_webhook_receiver`

07 Errors & retries

Errors are returned as `{ "error": ["code", ...] }`. Codes follow `<field>__<problem>` for validation and `<object>__<problem>` for business rules. The full catalogue (100 codes with recommended handling) is in `openapi/error-catalogue.md`.

Class	Retry?	What to do
400 validation — first_name__required, offer_price__invalid, price__required ...	No	Fix the request; surface field errors to the operator.
400 lifecycle — action not allowed for status	No	Re-read the deal; follow the lifecycle (§2). The local guard prevents most of these.
401 missing token · 403 api_token__invalid	No	Wrong or mistyped token, or sandbox token used against live (and vice-versa). Check <code>AMN_ENV</code> .
403 api_access__disabled	No	API access is not enabled for the account — contact Amen.
404	No	Object not found in this account/environment. Check the number.
429 rate_limit__exceeded	Yes	The client backs off exponentially with jitter and honours <code>Retry-After</code> . Batch bulk work.
5xx / network	Yes	Retried up to 3 times by the client; then alert.

```
try { await amen.deals.actions.approve(dealNo); }
catch (e) {
  if (e instanceof AmenApiError && e.has("price__required")) askSellerForFinalPrice();
  else if (e instanceof AmenLifecycleError) refreshDealAndRetryLater();
  else throw e;
}
```

Deeper: [docs/05-errors-and-retries.md](#) · [openapi/error-catalogue.md](#)

08 Mobile apps & web dashboards

Front ends never talk to `amnn.sa` directly. They talk to **your** back end, which holds the API token and uses one of the server-side reference clients.

Mobile / browser app —(your session token)—→ your back end —(X-API-Token)—→ `api.amnn.sa`
 Keychain · Keystore · any server stack escrow, payments,
 EncryptedSharedPreferences · from the kit webhooks
 flutter_secure_storage

iOS / Swift

Use `swift/'s AmenClient` on your server (Vapor/Hummingbird) or any other server stack. The app stores its own session token in the Keychain; see `swift/SampleApp/README.md`.

Android / Kotlin

Retrofit/OkHttp to your back end; session token in `EncryptedSharedPreferences` (Keystore-backed). Snippet in `kotlin/sample-android/`.

Flutter

`flutter/sample_app` shows the proxy pattern with `flutter_secure_storage`; `flutter/amen_client` is a pure-Dart package for your Dart back end or CLI.

Online payments from a mobile app

Your back end calls `make-payment-online` and returns the HyperPay `checkout_id` to the app; the app presents HyperPay's mobile SDK or web checkout with it. The back end learns the outcome from Amen's webhook and re-reads the deal.

Web dashboards (TypeScript / React / Next.js)

Put `AmenClient` behind a server route (`/api/amen/*` in Next.js route handlers, or an Express API) and have the React UI call that route. The `typescript/` client is typed from the spec, so the same types flow through to the UI layer. Certificate pinning to your own host is optional but recommended for mobile.

Deeper: `flutter/README.md` · `swift/SampleApp/README.md` · `kotlin/sample-android/README.md` · `typescript/README.md`

09 Go-live checklist

- ☐ Golden path and your own scenarios pass against the **sandbox** (integration tests green).
- ☐ A **live** token has been issued; it lives in a secret manager, not in code, images or CI logs.
- ☐ `AMN_ENV=live` is the only change between staging and production configuration.
- ☐ Webhook registered against your production URL over HTTPS; `secret_key` stored; verification, de-duplication and re-read implemented.
- ☐ Retries limited to 429/5xx; alerting on repeated 5xx and on webhook 401s.
- ☐ Money handled as decimal strings end-to-end; timestamps converted from epoch-ms.
- ☐ No API token in any mobile or browser artefact (grep your bundles).
- ☐ Operators can see `deal_number` and status in your admin tools for support conversations with Amen.
- ☐ You are subscribed to kit releases (`CHANGELOG.md`) so API changes reach you.

Deeper: `docs/06-security-checklist.md` · `docs/07-versioning.md`

10 Troubleshooting

Symptom	Check	Cause	Fix
<code>403 api_token__invalid</code> on every call	Base URL printed at startup	Sandbox token vs live URL (or vice-versa); mistyped token	Set <code>AMN_ENV</code> correctly / re-copy the token
<code>403 api_access__disabled</code>	—	API not enabled for this account	Contact Amen
<code>400</code> on <code>action/submit</code>	Deal has buyer and seller; product deal has delivery address	Parties/address missing	Call <code>parties/</code> and <code>delivery-address</code> first
<code>400 price__required</code> on approve	<code>offer_type</code>	Service deals need a final price	Pass <code>{ "price": "..." }</code>

Symptom	Check	Cause	Fix
Deal stuck in <code>payment_pending</code>	<code>allowed-payment-methods/</code> ; wallet balance	No wallet balance / online checkout not completed	Top up sandbox wallet or complete the HyperPay checkout; resume with the deal number
<code>AmenLifecycleError</code> in your code	Current deal status	Action called out of order	Follow §2; re-read the deal before acting
429	Request rate	Burst of calls	Let the client back off; batch work
Webhook signature never matches	Are you verifying the raw body?	Body re-serialised before verification; wrong secret	Verify bytes first, parse second; check <code>AMN_WEBHOOK_SECRET</code>
Dates in year 57,000	Timestamp handling	Epoch milliseconds treated as seconds	Use the model conversion / divide by 1000
Only some deals/customers processed	List call	Only page 0 read	Use the <code>iterAll / all()</code> helpers

Deeper: [docs/08-troubleshooting.md](#) · [docs/09-common-mistakes.md](#)

A Appendix — Endpoint map & deal statuses

Area	Endpoints (all under <code>/api/v1</code>)
Lookup	<code>GET allowed-country-codes/</code> · <code>cities</code> · <code>categories/</code> · <code>dispute-reasons/</code> · <code>dispute-resolution-reasons/</code> · <code>cancel-reasons/?party_type=</code>
Account	<code>GET account</code> · <code>GET/POST account/bank-accounts/</code> · <code>DELETE account/bank-accounts/{id}</code>
Customers	<code>GET/POST customers/</code> · <code>GET customers/{number}</code>
Deals	<code>GET/POST deals/</code> · <code>GET/PUT/DELETE deals/{n}</code> · <code>POST deals/{n}/parties/</code> · <code>delivery-address</code> · <code>billing-address</code> · <code>GET allowed-payment-methods/</code>
Deal actions	<code>POST deals/{n}/action/</code> <code>submit</code> · <code>approve</code> · <code>make-payment-wallet</code> · <code>make-payment-online</code> · <code>execution-start</code> · <code>execution-complete</code> · <code>complete</code> · <code>transfer-seller-amount</code> · <code>dispute</code> · <code>dispute-approve</code> · <code>dispute-decline</code> · <code>cancel</code>
Withdrawals	<code>GET/POST withdrawals/</code> · <code>GET withdrawals/{number}</code>
Webhooks	<code>GET/POST web-hooks/</code> · <code>DELETE web-hooks/{id}</code>

Deal statuses: `draft` · `requested` · `payment_pending` · `paid` · `executing` · `executed` · `completed` · `disputed` · `dispute_approved` · `canceled` · `payment_overdue`. **Offer types:** `product` (needs delivery address + delivery fee) · `service` (needs subject details; price fixed at approve). **Parties:** `buyer` · `seller` · `broker`.

B Appendix — Postman, versioning & support

Postman

Import `postman/Amen_API.postman_collection.json` and the *Amen* — *Sandbox* environment; set `apiToken`. Requests are ordered to follow the golden path and capture `customer_number`, `deal_number` and `webhook_id` into environment variables, so the whole lifecycle can be run with the Collection Runner.

Versioning

The API base path is `/api/v1`. Kit releases are tagged `v1.x.y`; `CHANGELOG.md` records the spec date each release was validated against. A weekly job compares the vendored spec with `docs.amnn.sa/openapi.yml` and opens an issue on drift, so watch the repository to be notified of API changes early. Because clients ignore unknown fields, additive changes never require action.

Getting help

- Check §10 and `openapi/error-catalogue.md` first — most questions are answered there.
- Open an issue on the kit repository for anything about the reference code or documentation (include stack, deal number, and the error codes returned — never your token).
- Contact the Amen integration team for account, token, wallet top-up and production questions.

Amen Platform API references: `docs.amnn.sa` · `sandbox-api.amnn.sa` · `api.amnn.sa` · Reference code: `github.com/KMG411/amen-integration-starter-kit` (MIT). This guide describes the kit at version 1.0; the repository is the source of truth where they differ.